

# Serial Communication

## Experiment Objective

This section will guide you through the basic usage of UART serial communication on the MicroROS-V2 control board. Through this tutorial, you will learn how to initialize UART1 and UART2, configure parameters such as baud rate, data bits, and stop bits, use a ring buffer to cache received data, and implement serial data send/receive echo functionality.

## Experiment Principle

### 1. Keywords

| Keyword     | Description                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| UART        | Universal Asynchronous Receiver/Transmitter, implements serial communication between devices through TX/RX lines, full-duplex operation |
| Baud Rate   | The serial communication rate, representing the number of symbols transmitted per second; common values include 9600 and 115200         |
| Data Bits   | The number of data bits in each data frame, usually 8 bits                                                                              |
| Parity      | A redundant bit used for error checking; options are none / odd / even parity                                                           |
| Stop Bits   | The bit(s) that mark the end of a data frame, usually 1 bit                                                                             |
| Ring Buffer | A fixed-size circular queue used to cache and read serial received data, preventing data loss                                           |

### 2. Technical Architecture

The program uses the UART peripheral driver of the ESP32-S3 to initialize two serial ports: UART1 (GPIO17 TXD, GPIO18 RXD) and UART2 (GPIO36 TXD, GPIO35 RXD), each independently configured with a baud rate of 115200, 8 data bits, no parity, and 1 stop bit. Each serial port has an independent FreeRTOS task responsible for receiving data, and the data is cached through a ring buffer. The main loop polls the buffer status of both serial ports; when data is available, it reads it byte by byte and sends it back over the same line (loopback), implementing the send/receive echo function.

Main component files:

- `main/main.c` — main program entry, initializes the serial ports and implements the echo logic
- `components/uart/uart1.c` / `uart1.h` — UART1 driver implementation and interface declarations
- `components/uart/uart2.c` / `uart2.h` — UART2 driver implementation and interface declarations
- `components/ring_buffer/ring_buffer.c` / `ring_buffer.h` — ring buffer implementation

# Experiment Steps

## 1. Hardware Preparation

| Hardware                        | Requirement                                                     |
|---------------------------------|-----------------------------------------------------------------|
| MicroROS-V2 control board       | ✔ Required                                                      |
| USB data cable                  | ✔ Required (for power supply and serial monitor)                |
| USB-to-TTL serial module        | ✔ Required (for connecting UART1 or UART2 to test send/receive) |
| Jumper wires (female-to-female) | ✔ Required (to connect the serial module to the control board)  |

## 2. Prerequisites

- 1. ESP32-IDF development environment set up
- 2. MicroROS-V2 control board connected to the PC via USB
- 3. Prepare a USB-to-TTL serial module (such as CH340, CP2102, etc.)

### Hardware wiring

Connect the USB-to-TTL serial module to the UART1 or UART2 pins of the control board:

| USB-to-TTL | MicroROS-V2 control board                |
|------------|------------------------------------------|
| TXD        | UART1_RXD (GPIO18) or UART2_RXD (GPIO35) |
| RXD        | UART1_TXD (GPIO17) or UART2_TXD (GPIO36) |
| GND        | GND                                      |

## 3. Compile and Flash

```
cd ~/esp/base_samples/uart
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

This example initializes both UART1 and UART2 at the same time, and the two serial ports work independently. After the program starts, you can see the receive task startup logs of UART1 and UART2 in the serial monitor. When data is sent to UART1 or UART2, the program sends the received data back unchanged.

## 4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

# Experiment Result

After correctly flashing and running the program, the serial monitor outputs the following startup information:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) UART2: Start Uart2_Rx_Task with core:1
I (xxx) UART1: Start Uart1_Rx_Task with core:1
```

Connect a USB-to-TTL serial module to the UART1 pins (GPIO17 TXD, GPIO18 RXD) and send data through a serial assistant; the control board will send the received data back unchanged. At the same time, you can see the received data length information in the logs:

```
I (xxx) MAIN: uart1_rx_len:5
```

## Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/uart/main/main.c`
- UART driver: `~/esp/base_samples/uart/components/uart/uart1.h`, `uart1.c`, `uart2.h`, `uart2.c`
- Ring buffer: `~/esp/base_samples/uart/components/ring_buffer/ring_buffer.h`, `ring_buffer.c`

## 1. Project Structure

```
uart/
├── CMakeLists.txt           # Top-level CMake project file
├── main/
│   ├── CMakeLists.txt      # main component build configuration
│   └── main.c              # Main program entry
├── components/
│   ├── ring_buffer/
│   │   ├── CMakeLists.txt
│   │   ├── ring_buffer.c    # Ring buffer implementation
│   │   └── ring_buffer.h    # Ring buffer interface
│   └── uart/
│       ├── CMakeLists.txt
│       ├── uart1.c          # UART1 driver implementation
│       ├── uart1.h          # UART1 interface declarations
│       ├── uart2.c          # UART2 driver implementation
│       └── uart2.h          # UART2 interface declarations
└── sdkconfig               # Project configuration
```

## 2. Main Program Logic

The execution flow of `app_main` in `main.c`:

1. Print the startup messages `"hello yahboom"` and `"Nice to meet you!"`
2. Call `uart2_Init()` and `uart1_Init()` in turn to initialize the two serial ports
3. Enter the main loop, polling to check whether UART1 and UART2 have incoming data

- When data is detected, read it byte by byte and send it back over the same line (loopback)
- Print the received data length log after each send-back completes

### 3. Key Components

#### UART1 Driver (uart1.h / uart1.c)

##### Pin definitions:

Parameter definition file: `~/esp/base_samples/uart/components/uart/uart1.h`

```
#define UART1_GPIO_TXD  GPIO_NUM_17    // UART1 transmit pin
#define UART1_GPIO_RXD  GPIO_NUM_18    // UART1 receive pin
#define RX1_BUF_SIZE    1024           // Receive buffer size
```

##### Serial port parameter configuration:

```
const uart_config_t uart_config = {
    .baud_rate    = 115200,              // Baud rate 115200
    .data_bits    = UART_DATA_8_BITS,    // 8 data bits
    .parity       = UART_PARITY_DISABLE, // No parity
    .stop_bits    = UART_STOP_BITS_1,    // 1 stop bit
    .flow_ctrl    = UART_HW_FLOWCTRL_DISABLE, // No hardware flow control
    .source_clk   = UART_SCLK_DEFAULT,   // Default clock source
};
```

##### Initialization flow (Uart1\_Init):

- Call `uart_driver_install(UART_NUM_1, RX1_BUF_SIZE * 2, 0, 0, NULL, 0)` to install the UART1 driver
- Call `uart_param_config(UART_NUM_1, &uart_config)` to configure the serial port parameters
- Call `uart_set_pin(UART_NUM_1, UART1_GPIO_TXD, UART1_GPIO_RXD, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE)` to set the GPIO pins
- Initialize the ring buffer `RingBuffer_Init(&uart1_ringbuf, RX1_BUF_SIZE)`
- Create the receive task `Uart1_Rx_Task` (stack size 5 KB, priority 5)

##### Receive task (Uart1\_Rx\_Task):

- Loop calling `uart_read_bytes(UART_NUM_1, temp_data, temp_len, 1 / portTICK_PERIOD_MS)` to read data
- Store each read byte into the ring buffer via `RingBuffer_Push`

##### API functions:

| Function                                       | Description      |
|------------------------------------------------|------------------|
| <code>void Uart1_Init(void)</code>             | Initialize UART1 |
| <code>int Uart1_Send_Byte(uint8_t data)</code> | Send one byte    |

| Function                                                      | Description                              |
|---------------------------------------------------------------|------------------------------------------|
| <code>int Uart1_Send_Data(uint8_t* data, uint16_t len)</code> | Send a series of data                    |
| <code>uint16_t Uart1_Available(void)</code>                   | Return the number of buffered data items |
| <code>uint8_t Uart1_Read(void)</code>                         | Read one byte from the buffer            |
| <code>void Uart1_Clean_Buffer(void)</code>                    | Clear the buffered data                  |

## UART2 Driver (uart2.h / uart2.c)

### Pin definitions:

```
#define UART2_GPIO_TXD  GPIO_NUM_36  // UART2 transmit pin
#define UART2_GPIO_RXD  GPIO_NUM_35  // UART2 receive pin
#define RX2_BUF_SIZE    1024         // Receive buffer size
```

The configuration and initialization flow of UART2 is exactly the same as UART1, the only difference being that `UART_NUM_2` is used as the serial port number. The API functions are correspondingly named `Uart2_Init`, `Uart2_Send_Byte`, `Uart2_Send_Data`, `Uart2_Available`, `Uart2_Read`, `Uart2_Clean_Buffer`.

## Ring Buffer (ring\_buffer.h / ring\_buffer.c)

The ring buffer is used to cache serial received data and avoid data loss.

### Data structure:

```
typedef struct {
    uint8_t *buffer;    // Buffer
    uint16_t head;      // Head pointer
    uint16_t tail;      // Tail pointer
    uint16_t size;      // Size of the ring queue (actual capacity = size - 1)
} Ring_Buffer_t;
```

### Core functions:

| Function                                                                     | Description                                                                     |
|------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>void RingBuffer_Init(Ring_Buffer_t* ringBuf, uint16_t capacity)</code> | Initialize the ring buffer (allocates capacity+1 bytes of memory)               |
| <code>void RingBuffer_Push(Ring_Buffer_t* ringBuf, uint8_t value)</code>     | Write to the tail (automatically pops the head element when the buffer is full) |
| <code>uint8_t RingBuffer_Pop(Ring_Buffer_t* ringBuf)</code>                  | Pop the head element                                                            |
| <code>uint16_t RingBuffer_Get_Used_Count(Ring_Buffer_t* ringBuf)</code>      | Get the number of stored bytes                                                  |
| <code>uint16_t RingBuffer_Get_Free_Count(Ring_Buffer_t* ringBuf)</code>      | Get the remaining space                                                         |

| Function                                                         | Description                                            |
|------------------------------------------------------------------|--------------------------------------------------------|
| <code>uint8_t RingBuffer_IsEmpty(Ring_Buffer_t* ringBuf)</code>  | Check whether the queue is empty                       |
| <code>uint8_t RingBuffer_IsFull(Ring_Buffer_t* ringBuf)</code>   | Check whether the queue is full                        |
| <code>void RingBuffer_Clean_Queue(Ring_Buffer_t* ringBuf)</code> | Clear all data                                         |
| <code>void RingBuffer_Reset(Ring_Buffer_t* ringBuf)</code>       | Reset the queue (pointers reset to zero, data cleared) |

In addition, the ring buffer also provides multi-byte type read/write functions for conveniently reading and writing data types such as `Short`, `Long`, and `Float`:

| Function                                                                                         | Description                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>uint8_t RingBuffer_ReadByte(Ring_Buffer_t* ringBuf)</code>                                 | Read one byte                     |
| <code>void RingBuffer_ReadByteArray(Ring_Buffer_t* ringBuf, uint8_t* dest, uint16_t size)</code> | Read a byte array                 |
| <code>int16_t RingBuffer_ReadShort(Ring_Buffer_t* ringBuf)</code>                                | Read a signed Short (2 bytes)     |
| <code>uint16_t RingBuffer_ReadUShort(Ring_Buffer_t* ringBuf)</code>                              | Read an unsigned Short (2 bytes)  |
| <code>int32_t RingBuffer_ReadLong(Ring_Buffer_t* ringBuf)</code>                                 | Read a signed Long (4 bytes)      |
| <code>uint32_t RingBuffer_ReadULong(Ring_Buffer_t* ringBuf)</code>                               | Read an unsigned Long (4 bytes)   |
| <code>float RingBuffer_ReadFloat(Ring_Buffer_t* ringBuf)</code>                                  | Read a Float (4 bytes)            |
| <code>void RingBuffer_WriteByte(Ring_Buffer_t* ringBuf, uint8_t value)</code>                    | Write one byte                    |
| <code>void RingBuffer_WriteByteArray(Ring_Buffer_t* ringBuf, uint8_t* src, uint16_t size)</code> | Write a byte array                |
| <code>void RingBuffer_WriteShort(Ring_Buffer_t* ringBuf, int16_t value)</code>                   | Write a Short (2 bytes)           |
| <code>void RingBuffer_WriteUShort(Ring_Buffer_t* ringBuf, uint16_t value)</code>                 | Write an unsigned Short (2 bytes) |
| <code>void RingBuffer_WriteLong(Ring_Buffer_t* ringBuf, int32_t value)</code>                    | Write a Long (4 bytes)            |
| <code>void RingBuffer_WriteULong(Ring_Buffer_t* ringBuf, uint32_t value)</code>                  | Write an unsigned Long (4 bytes)  |
| <code>void RingBuffer_WriteFloat(Ring_Buffer_t* ringBuf, float value)</code>                     | Write a Float (4 bytes)           |

| Function                                                                                 | Description              |
|------------------------------------------------------------------------------------------|--------------------------|
| <code>uint8_t RingBuffer_GetValue_ByIndex(Ring_Buffer_t* ringBuf, uint16_t index)</code> | Read an element by index |

## Common Problems

| Problem                    | Cause                                 | Solution                                                                                                                            |
|----------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| No serial output           | Wrong wiring of the USB-to-TTL module | Check whether TXD and RXD are cross-connected (TXD to the other side's RXD, RXD to the other side's TXD), and confirm GND is common |
| Garbled received data      | Baud rate mismatch                    | Confirm the USB-to-TTL module baud rate is set to 115200                                                                            |
| Unstable data send/receive | No common ground or wiring too long   | Make sure the control board and the serial module share GND, and shorten the wiring distance                                        |